

Introduction to R

Session 08: Probability and Simulation

Álvaro Pérez

Instituto Tecnológico Autónomo de México

Spring 2025

Overview of Today's Session

Probability

Univariate Distributions

Multivariate Distributions

Statistical Terms

Tests

Example

Important Notation

- ▶ Greek letters (e.g., μ) are the truth (i.e., parameters of the true DGP)
- ▶ Greek letters with hats (e.g., $\hat{\mu}$) are estimates (i.e., what we *think* the truth is)
- ▶ Non-Greek letters (e.g., X) denote sample/data
- ▶ Non-Greek letters with lines on top (e.g., $\bar{X}$) denote calculations from the data (e.g., $\bar{X} = \frac{1}{N} \sum_i X_i$).
- ▶ We want to estimate the truth, with some calculation from the data ($\hat{\mu} = \bar{X}$)
- ▶ Data $\longrightarrow$ Calculations $\longrightarrow$ Estimate $\xrightarrow{\text{Hopefully}}$ Truth
- ▶ Example: $X \longrightarrow \bar{X} \longrightarrow \hat{\mu} \xrightarrow{\text{Hopefully}} \mu$

Important Notation

Example with a coin toss

- ▶ μ denotes the true probability a coin lands head ($\frac{1}{2}$ if the coin is fair)
- ▶ $\hat{\mu}$ is our estimator of the probability a coin lands head
- ▶ X is the data we gather from tossing a coin 500 times
- ▶ $\bar{X}$ is the proportion of times the coin lands head
- ▶ Data from coin tosses $\longrightarrow$ Calculate proportion of heads $\longrightarrow$ Estimator for the probability of heads $\xrightarrow{\text{Hopefully}}$ True probability
- ▶ $X \longrightarrow \bar{X} \longrightarrow \hat{\mu} \xrightarrow{\text{Hopefully}} \mu$

For any constants a and b and random variables X and Y :

- ▶ $\mathbb{E}[aX + b] = a\mathbb{E}[X] + b$
- ▶ $\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$
- ▶ $V(aX + b) = a^2 V(X)$
- ▶ $\text{Cov}(X, Y) := \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y]$
- ▶ $\text{Cor}(X, Y) := \frac{\text{Cov}(X, Y)}{\sqrt{V(X)}\sqrt{V(Y)}} \in [-1, 1]$
- ▶ $V(X + Y) = V(X) + V(Y) + 2\text{Cov}(X, Y)$

Uniform Distribution

A **univariate distribution** is the distribution of a single variable. The **uniform distribution** is the simplest distribution. All numbers in the range have an equal probability of being sampled.

- **Continuous distribution**: Use `runif()` to sample from a continuous uniform distribution.

```
1 u <- runif(100000, min = 0, max = 1)
2
3 # plot to visualise
4 ggplot() +
5   geom_histogram(aes(u), binwidth = 0.05, boundary =
6     0,
7                       fill = "white", colour = "black")
```

- **Discrete:** Use `sample()` to sample from a discrete distribution.

The code below simulates rolling a 6-sided die 10000 times. We set `replace` to `TRUE` so that each event is independent.

```
1 rolls <- sample(1:6, 10000, replace = TRUE)
2
3 # plot the results
4 ggplot() +
5   geom_histogram(aes(rolls), binwidth = 1,
6                  fill = "white", color = "black")
```

The binomial distribution is useful for modelling binary data, where each observation can have one of two outcomes, like success/failure, yes/no or head/tails.

rbinom(n, size, prob)

The rbinom function will generate a random binomial distribution.

- ▶ `n` = number of observations
- ▶ `size` = number of trials
- ▶ `prob` = probability of success on each trial

`rnorm(n, mean, sd)`

We can simulate a normal distribution of size n if we know the mean and standard deviation (sd). A density plot is usually the best way to visualise this type of data if your n is large.

The Poisson distribution is useful for modelling events, like how many times something happens over a unit of time, as long as the events are independent

rpois(n, lambda)

The rpois function will generate a random Poisson distribution.

- ▶ n = number of observations
- ▶ λ = the mean number of events per observation

Poisson Distribution

Let's say we want to model how many texts you get each day for a whole. You know that you get an average of 20 texts per day. So we set $n = 365$ and $\lambda = 20$. Lambda is a parameter that describes the Poisson distribution, just like mean and standard deviation are parameters that describe the normal distribution.

```
1 texts <- rpois(n = 365, lambda = 20)
2
3 ggplot() +
4   geom_histogram(
5     aes(texts),
6     binwidth = 1,
7     fill = "white",
8     color = "black"
9   )
```

Bivariate Normal

- ▶ A **bivariate normal** distribution is two normally distributed vectors that have a specified relationship, or **correlation** to each other.
- ▶ What if we want to sample from a population with specific relationships between variables? We can sample from a bivariate normal distribution using `mvnrm()` from the MASS package.
- ▶ You need to know how many observations you want to simulate (n) the means of the two variables (μ) and you need to calculate a **covariance matrix** (σ) from the correlation between the variables (ρ) and their standard deviations (sd).

Bivariate Normal

```
1  n    <- 1000 # number of random samples
2  # name the mu values to give the resulting columns
   names
3  mu    <- c(x = 10, y = 20) # the means of the samples
4  sd <- c(5, 6)    # the SDs of the samples
5
6  rho <- 0.5 # population correlation between the two
   variables
7
8  # correlation matrix
9  cor_mat <- matrix(c( 1, rho,
10                      rho, 1), 2)
11
12 # create the covariance matrix
13 sigma <- (sd %*% t(sd)) * cor_mat
14
15 # sample from bivariate normal distribution
16 bvn <- MASS::mvrnorm(n, mu, sigma)
```

Multivariate Normal

You can generate more than 2 correlated variables, but it gets a little trickier to create the correlation matrix.

```
1  n      <- 200 # number of random samples
2  mu     <- c(x = 10, y = 20, z = 30) # the means of the
   samples
3  sd <- c(8, 9, 10) # the SDs of the samples
4  rho1_2 <- 0.5 # correlation between x and y
5  rho1_3 <- 0 # correlation between x and z
6  rho2_3 <- 0.7 # correlation between y and z
7  # correlation matrix
8  cor_mat <- matrix(c(      1, rho1_2, rho1_3,
9                          rho1_2,      1, rho2_3,
10                         rho1_3, rho2_3,      1), 3)
11
12 sigma <- (sd %*% t(sd)) * cor_mat
13 bvn3 <- MASS::mvrnorm(n, mu, sigma)
14 cor(bvn3) # check correlation matrix
```

Alternatively, you can use the package **faux** to generate any number of correlated variables. It also has a function for checking the parameters of your new simulated data (`check_sim_stats()`).

```
1  bvn3 <- rnorm_multi(  
2    n = n,  
3    vars = 3,  
4    mu = mu,  
5    sd = sd,  
6    r = c(rho1_2, rho1_3, rho2_3),  
7    varnames = c("x", "y", "z")  
8  )  
9  
10 check_sim_stats(bvn3)
```

- ▶ **Effect:** The **effect** is some measure of your data. For example, if you flipped a coin 100 times and it landed heads 66 times, the effect would be 66/100.
- ▶ **P-value:** The **p-value** of a test is the probability of seeing an effect at least as extreme as what you have, if the real effect was the value you are testing against (e.g., a null effect).
- ▶ **Alpha:** If you are using null hypothesis significance testing (**NHST**), then you need to decide on a cutoff value (**alpha**) for making a decision to reject the null hypothesis. We call p-values below the alpha cutoff **significant**.

- ▶ **False Positive/Negative:**
 - ▶ **False positive:** When a test concludes there is an effect when there really is no effect (**Type I Error**).
 - ▶ **False negative:** When a test concludes there is no effect when there really is an effect (**Type II Error**).
- ▶ **Power and SESOI:** **Power** is equal to 1 minus beta (i.e., the true positive rate), and depends on the effect size, how many samples we take (n), and what we set alpha to. For any test, if you specify all but one of these values, you can calculate the last. The effect size you use in power calculations should be the smallest effect size of interest (**SESOI**).
- ▶ **Confidence Intervals:** The **confidence interval** is a range around some value (such as a mean) that has some probability of containing the parameter, if you repeated the process many times.

binom.test(x, n, p)

You can test a binomial distribution against a specific probability using the exact binomial test.

- ▶ x = the number of successes
- ▶ n = the number of trials
- ▶ p = hypothesised probability of success

Here we can test a series of 10 coin flips from a fair coin and a biased coin against the hypothesised probability of 0.5 (even odds).

```
1 n <- 10
2 fair_coin <- rbinom(1, n, 0.5)
3 biased_coin <- rbinom(1, n, 0.6)
4
5 binom.test(fair_coin, n, p = 0.5)
6 binom.test(biased_coin, n, p = 0.5)
```

Exact binomial test

Sampling function

To estimate these rates, we need to repeat the sampling above many times. A function is ideal for repeating the exact same procedure over and over. Set the arguments of the function to variables that you might want to change. Here, we want to estimate power for:

- ▶ different sample sizes (n)
- ▶ different effects (bias)
- ▶ different hypothesised probabilities (p , defaults to 0.5)

```
1 sim_binom_test <- function(n, bias, p = 0.5) {  
2   coin <- rbinom(1, n, bias)  
3   btest <- binom.test(coin, n, p)  
4   btest$p.value  
5 }  
6 sim_binom_test(100, 0.6)
```

Exact binomial test

Calculate power

You can use the `replicate()` function to run it many times and save all the output values. You can calculate the power of your analysis by checking the proportion of your simulated analyses that have a p-value less than your alpha (the probability of rejecting the null hypothesis when the null hypothesis is true).

```
1 my_reps <- replicate(1e4, sim_binom_test(100, 0.6))
2
3 alpha <- 0.05 # this does not always have to be 0.05
4
5 mean(my_reps < alpha)
```

t.test(x, y, alternative, mu, paired)

Use a t-test to compare the mean of one distribution to a null hypothesis (one-sample t-test), compare the means of two samples (independent-samples t-test), or compare pairs of values (paired-samples t-test).

You can run a one-sample t-test comparing the mean of your data to mu.

```
1 sim_norm <- rnorm(100, 0.5, 1)
2 t.test(sim_norm, mu = 0)
```

T-test

Sampling function

We can use the `names()` function to find out the names of all the `t.test` parameters and use this to just get one type of data, like the test statistic (e.g., `t-value`).

```
1 names(t_ind)
2 t_ind$statistic
```

If you want to run the simulation many times and record information each time, first you need to turn your simulation into a function.

T-test

Calculate power

Now replicate the simulation 1000 times.

```
1 my_reps <- replicate(1e4, sim_t_ind(100, 0.7, 1, 0.5,  
  1))  
2  
3 alpha <- 0.05  
4 power <- mean(my_reps > alpha)  
5 power
```

Compare your power estimate from simulation to a power calculation using `power.t.test()`. Here, delta is the difference between m1 and m2 above.

```
1 power.t.test(n = 100,  
2             delta = 0.2,  
3             sd = 1,  
4             sig.level = alpha,  
5             type = "two.sample")
```

You can test if continuous variables are related to each other using the `cor()` function. Let's use `rnorm_multi()` to make a quick table of correlated values.

```
1 dat <- rnorm_multi(  
2   n = 100,  
3   vars = 2,  
4   r = -0.5,  
5   varnames = c("x", "y")  
6 )  
7  
8 cor(dat$x, dat$y)
```

`cor()` defaults to Pearson's correlations. Set the `method` argument to use Kendall or Spearman correlations.

Correlation

Sampling function

Create a function that creates two variables with n observations and r correlation. Use the function `cor.test()` to give you p-values for the correlation.

```
1 sim_cor_test <- function(n = 100, r = 0) {  
2   dat <- rnorm_multi(  
3     n = n,  
4     vars = 2,  
5     r = r,  
6     varnames = c("x", "y")  
7   )  
8  
9   ctest <- cor.test(dat$x, dat$y)  
10  ctest$p.value  
11 }
```

Now replicate the simulation 1000 times.

```
1 my_reps <- replicate(1e4, sim_cor_test(50, 0.5))
2
3 alpha <- 0.05
4 power <- mean(my_reps < alpha)
5 power
6 # Compare to the value calculated by the pwr package.
7 pwr::pwr.r.test(n = 50, r = 0.5)
```

This example uses the Growth Chart Data Tables from the US CDC. The data consist of height in centimeters for the z-scores of -2, -1.5, -1, -0.5, 0, 0.5, 1, 1.5, and 2 by sex (1=male; 2=female) and half-month of age (from 24.0 to 240.5 months).

We have to:

1. Load and wrangle with the data
2. Plot
3. Simulate a population
4. Analyse simulated data
5. Replicate simulation
6. One-tailed prediction

Go to the R script of this session